

FINAL YEAR PROJECT PROPOSAL

PeerLAN

Design, Implementation, and Evaluation of a Zero-Install, Offline,
End-to-End Encrypted Peer-to-Peer File Transfer System for Closed
Networks

B.Sc. Computer Science, Semester VI

Submitted by

Hariharan S

Roll No. 222407723

Submitted to

Mrs. M. Subbulakshmi, M.Sc., M.Phil., Ph.D.

Head of the Department, Department of Computer Science

6 October 2026

PeerLAN: Design, Implementation, and Evaluation of a Zero-Install, Offline, End-to-End Encrypted Peer-to-Peer File Transfer System for Closed Networks

Final Year Project Proposal

Student	Hariharan S
Roll No.	222407723
Programme	B.Sc. Computer Science, Semester VI
Submitted To	Mrs. M. Subbulakshmi, M.Sc., M.Phil., Ph.D. Head of the Department, Department of Computer Science
Date	6 October 2026

Table of Contents

1. Introduction (Problem Scenario and PeerLAN's Role)
2. Aims and Objectives
3. Expected Outcomes and Deliverables
4. Project Risks, Threats, and Contingency Plans
5. Methodology
6. Resource Requirements
7. Work Breakdown Structure
8. Milestones
9. Project Gantt Chart
10. Conclusion
11. References

1. Introduction

1.1 Background

Moving a file between two nearby devices is a routine task, yet the common solutions are poorly suited to closed or restricted networks. Cloud storage and messaging apps need internet access and keep copies on external servers. Pen drives are slow and leave untracked copies. Many local-sharing tools must be installed on every device.

Modern browsers now include **WebRTC**, which allows direct device-to-device connections, and the **Web Crypto API**, which provides standard encryption. Together they make it possible to build a secure local transfer system that needs nothing more than a browser on the receiving side.

1.2 Problem Scenario

Consider two devices on the same local network (a shared WiFi, a wired LAN, or a phone hotspot) where internet access is unavailable, restricted, or not trusted. **Device A** needs to hand a sensitive file to **Device B**. Today the options are poor:

Current option	Weakness in this scenario
Cloud storage or email	Needs internet; copy stored on a third-party server
Messaging apps	Needs internet; file passes through external servers
Pen drive	Slow; malware risk; untracked copies
Installed sharing apps	Requires installation on both devices

Core problem: there is no simple way to send a sensitive file between two devices on a local network that (a) works with the internet disconnected, (b) needs no software installed on the receiving device, and (c) keeps the file encrypted even from the machine hosting the session.

1.3 Existing Solutions and the Gap

Tool	Type	Observation
Snapdrop / PairDrop	Open-source browser-based sharing	General-purpose; not built around a controlled, confidential transfer workflow
LocalSend	Open-source LAN sharing application	Requires an installed app on the devices
Keet (Holepunch)	Peer-to-peer chat app with file sharing	Full chat platform installed on both devices; DHT-based peer discovery; behaviour without internet to be tested
Cloud and messaging apps	WhatsApp, Google Drive, email	Need internet; files stored on third-party servers

Each observation will be confirmed through hands-on testing during the project.

This project does not claim to invent device-to-device file transfer. It targets a specific gap: a **zero-install-receiver, fully offline, approval-controlled, end-to-end encrypted** transfer, supported by measured evaluation.

1.4 PeerLAN's Role in Problem Management

Problem	How PeerLAN handles it
Internet unavailable or untrusted	Works entirely on the local network, with no internet needed
Software installation on the receiver	Receiver uses only a web browser
File readable by the host or an eavesdropper	Passphrase-based end-to-end encryption on top of WebRTC's own encryption
Unwanted devices joining	Expiring single-use session codes plus sender approval for every receiver
Network blocks direct device traffic	Encrypted relay fallback through the host
Hard to trust the result	Integrity check (SHA-256) and per-chunk authentication detect tampering

2. Aims and Objectives

2.1 Aim

To design, build, and evaluate **PeerLAN**, a secure peer-to-peer file transfer system that works offline on a local network and requires no installation on the receiving device.

2.2 Objectives

1. Build a working offline peer-to-peer file transfer system for local networks.
2. Ensure files are end-to-end encrypted and tamper-evident.
3. Control access through sender approval and short-lived sessions.
4. Allow the receiver to use only a web browser, with no installation.
5. Provide a fallback so transfers still work when direct connections are blocked.
6. Evaluate performance, compatibility, and security with real tests, and compare with existing tools.
7. Document the design, threat model, and results.

3. Expected Outcomes and Deliverables

3.1 Expected Outcomes

- A tested system where **Device A** sends a file to **Device B** with the internet disconnected.
- Measured evidence of speed, encryption overhead, and browser compatibility.
- Packet-capture evidence that traffic is unreadable and stays on the local network.
- A comparison showing where PeerLAN performs better or differently from existing tools.
- A reusable open-source base for future work.

3.2 Deliverables

#	Deliverable
1	Working application with source code (single-executable host and browser client)
2	Software Requirements Specification (SRS), threat model, and test report
3	User guide and demo video
4	Final project report and presentation

4. Project Risks, Threats, and Contingency Plans

4.1 Project Risks

Risk	Likelihood	Impact	Contingency plan
Network blocks device-to-device traffic (client isolation)	Medium	High	Encrypted relay fallback through the host; test early on the target network; use phone hotspot for demonstrations
WebRTC learning curve slows progress	Medium	Medium	Build a minimal two-device prototype first; add features only after each step works
Browser differences, especially on iPhone	Medium	Medium	Define a supported-browser list; test early and document limits
Scope growth	High	Medium	Fix a "Must" feature list; optional features (relay, audit log, resume) only after testing passes
Unsigned executable flagged by antivirus	Medium	Low	Provide a source-run version as backup; explain in the user guide
Browser rejects the local HTTPS certificate	High	Low	Generate a local certificate; document the one-time warning
Individual project: one person carries all work; exams or illness may delay	Medium	Medium	Start early (October), keep a buffer in the schedule, cut optional features first

4.2 Security Threats

Threat	Planned protection
Eavesdropping on the network	WebRTC DTLS plus AES-GCM encryption with a passphrase-derived key
Guessing a session code	Random, short-lived, single-use code; attempt limits; sender approval
Tampering with data	Per-chunk authentication and final SHA-256 verification
Curious or compromised host	Key never leaves the browsers; host forwards only encrypted data

Accepted limitations: PeerLAN protects data *in transit*. It cannot stop a receiver from copying a file after saving it or protect a device that is already compromised.

5. Methodology

5.1 Development Approach

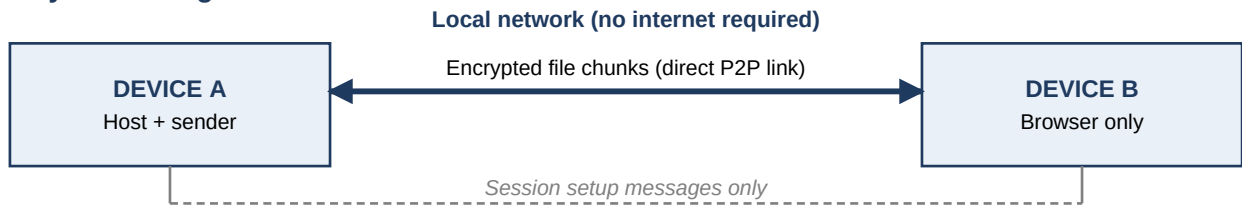
Iterative and incremental development with prototyping. The system is built in small working steps, and each phase ends with a demonstration. Features are prioritised as Must, Should, and Could, so the core is finished before extras begin.

5.2 Phases

1. **Study and feasibility:** test existing tools, confirm network behaviour on a hotspot and the target WiFi, define the threat model.
2. **Connection prototype:** two devices connect and exchange data through WebRTC.
3. **Host application and sessions:** HTTPS server, session codes, expiry, sender approval.
4. **Transfer engine:** chunked, reliable file transfer with backpressure and progress reporting.
5. **Security layer:** key derivation, authenticated encryption, integrity checking.
6. **Fallback and extras:** encrypted relay mode, optional audit log.
7. **Testing and evaluation:** benchmarks, compatibility, security verification, tool comparison.

8. **Packaging and documentation:** single executable, user guide, final report.

5.3 System Design



Workflow: create a session, share code and passphrase, receiver joins, sender approves, direct connection forms, file is encrypted and sent in chunks, integrity is verified, session closes.

5.4 Technology

Purpose	Technology
Language	JavaScript (Node.js and browser)
Direct connection	WebRTC data channels
Session setup	WebSocket over HTTPS
Encryption	Web Crypto API (AES-GCM, PBKDF2, SHA-256)
Optional audit log	SQLite (metadata only)
Packaging	Node.js Single Executable Application

5.5 Evaluation Method

Test	Output
Transfer works with the internet disconnected	Video evidence
Transfer speed (1 MB to 500 MB) and encryption overhead	Tables and graphs
Browser and device compatibility	Compatibility matrix
Wireshark packet capture	Proof traffic is unreadable and stays on the local network
Comparison with existing tools	Setup effort, success rate, and speed

6. Resource Requirements

Category	Requirement	Cost
Hardware	Development laptop (Windows or Linux); one or two phones for testing; optional second laptop; optional Raspberry Pi as an always-on host	Nil (own devices)
Network	WiFi router or phone hotspot (internet not required); access to the target WiFi for real-world testing, with permission	Nil
Software	Node.js, VS Code, Git/GitHub, mkcert or OpenSSL, Wireshark, a packaging tool (Node SEA or pkg)	Nil (all free)
Skills to acquire	WebRTC, Web Crypto API, WebSocket, basic networking and TLS	Self-study
Human	One student developer (individual project), guidance from the project guide	N/A
Other	Volunteers for usability testing, using non-confidential sample files only	Nil

Total estimated cost: nil.

7. Work Breakdown Structure

The WBS splits the whole project into small, manageable tasks so nothing is forgotten and progress can be tracked.

ID	Work package	Main tasks
1	Project management	1.1 Proposal; 1.2 SRS; 1.3 Threat model; 1.4 Progress tracking in Git
2	Study and feasibility	2.1 Test existing tools (Snapdrop, PairDrop, LocalSend, Keet); 2.2 Hotspot and WiFi reachability test; 2.3 Learn WebRTC and Web Crypto
3	Connection prototype	3.1 Signaling over WebSocket; 3.2 Two-device data channel; 3.3 Text exchange demo

ID	Work package	Main tasks
4	Host application	4.1 HTTPS server and local certificate; 4.2 IP detection, link and QR code; 4.3 Session codes, expiry, rate limits; 4.4 Sender approval
5	Transfer engine	5.1 Chunking and backpressure; 5.2 Progress, speed, and time remaining; 5.3 Streaming save to disk; 5.4 Error handling
6	Security layer	6.1 PBKDF2 key derivation; 6.2 AES-GCM per chunk; 6.3 SHA-256 file verification; 6.4 Key cleanup
7	Fallback and extras (optional)	7.1 Encrypted relay mode; 7.2 Audit log; 7.3 Resume interrupted transfer
8	Testing and evaluation	8.1 Functional tests; 8.2 Benchmarks; 8.3 Compatibility matrix; 8.4 Wireshark verification; 8.5 Tool comparison; 8.6 Usability test
9	Packaging	9.1 Single executable (Windows and Linux); 9.2 Clean-machine test
10	Documentation and presentation	10.1 User guide; 10.2 Deployment guide; 10.3 Demo video; 10.4 Final report; 10.5 Presentation

8. Milestones

Milestones are checkpoints with a clear pass/fail result. They show the guide and HOD measurable progress, and warn early if the project is slipping. Dates are targets for October 2026 to April 2027.

#	Milestone	Success criterion	Target
M1	Feasibility complete	Existing tools tested; hotspot reachability confirmed; SRS and threat model ready	End Nov
M2	Connection demo	Two devices connect through WebRTC and exchange messages	End Dec
M3	File transfer working	A file transfers between Device A and Device B with matching hash	Mid Jan
M4	Secure transfer complete	Approval flow, passphrase encryption, and integrity check working	End Feb
M5	Evaluation complete	Benchmarks, compatibility matrix, Wireshark evidence, and tool comparison done	End Mar
M6	Product ready	Packaged executable tested on a clean machine; user guide and demo video done	Mid Apr
M7	Final submission	Final report and presentation delivered	End Apr

9. Project Gantt Chart

The Gantt chart shows when each work package happens and how they overlap, and it shows the plan fits inside the Semester VI window (mid-December to April), with a head start in October and November.

WBS	Activity	Oct 26	Nov 26	Dec 26	Jan 27	Feb 27	Mar 27	Apr 27
1	Project management							
2	Study and feasibility							
3	Connection prototype							
4	Host application and sessions							
5	Transfer engine							
6	Security layer							
7	Fallback and extras (optional)							
8	Testing and evaluation							
9	Packaging							
10	Documentation and presentation							
	Milestone reached		M1	M2	M3	M4	M5	M6, M7

Buffer: Mid-March to April includes time for fixes and corrections. If any phase slips, optional features (WBS 7) are cut first. Security and testing are never cut.

10. Conclusion

File transfer between devices on a closed network is a real need that current cloud, messaging, and installed tools handle poorly. PeerLAN proposes a focused answer: a **zero-install-receiver, offline, end-to-end encrypted** peer-to-peer transfer system with sender approval and an encrypted fallback.

Although device-to-device sharing tools already exist, this project's value lies in its security-focused design, its offline-first deployment, and above all its **measured, documented evaluation** against those tools. The plan is realistic: it starts early, builds in small tested steps, prioritises core features, and fits within the Semester VI timeline with a buffer. The result will be a working, tested, and reusable system, developed as an individual project.

11. References

1. W3C, *WebRTC 1.0: Real-Time Communication Between Browsers*, W3C Recommendation. <https://www.w3.org/TR/webrtc/>
2. W3C, *Web Cryptography API*, W3C Recommendation. <https://www.w3.org/TR/WebCryptoAPI/>
3. IETF, *RFC 8831: WebRTC Data Channels*, 2021.
4. IETF, *RFC 8445: Interactive Connectivity Establishment (ICE)*, 2018.
5. IETF, *RFC 6455: The WebSocket Protocol*, 2011.
6. M. Dworkin, *NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, 2007.
7. M. Turan et al., *NIST SP 800-132: Recommendation for Password-Based Key Derivation, Part 1: Storage Applications*, 2010.
8. Mozilla, *MDN Web Docs: WebRTC API, SubtleCrypto, and File System Access API*. <https://developer.mozilla.org/>
9. Node.js, *Single Executable Applications*, Node.js documentation. <https://nodejs.org/api/single-executable-applications.html>
10. Snapdrop. <https://github.com/RobinLinus/snapdrop>
11. PairDrop. <https://github.com/schlagmichdoch/PairDrop>
12. LocalSend. <https://localsend.org>
13. Holepunch, Keet. <https://keet.io>

Submitted by Hariharan S (Roll No. 222407723) as an individual Final Year Project proposal to Mrs. M. Subbulakshmi, M.Sc., M.Phil., Ph.D., Head of the Department, Department of Computer Science.